

Coursework 1 - **Deadline: Friday, 13 February 2026 at 1 pm.**

General instructions - please read carefully

The goal of this coursework is to analyse a dataset using several tools and algorithms introduced in the lectures, which you have also studied in detail through the weekly Python notebooks containing the computational tasks. You will solve the tasks in this coursework using Python.

You are allowed to use:

- Python code that you have developed in your coding tasks;
- any other basic mathematical functions contained in `numpy`;
- `pandas` to build and clean up data tables;
- `matplotlib` and `seaborn` for visualisation.

You are **not** allowed to use:

- any model-level or automatic differentiation Python packages (e.g., `scikit-learn`, `statsmodels`, `PyTorch`, `JAX`, `Tensorflow/Keras`, etc);
- ready-made code found anywhere online;
- conversational AI tools such as ChatGPT, Copilot, Gemini etc. to generate code and written answers.

Needless to say, your submission must be your own individual work: You may discuss the analysis with your colleagues but code, written answers, figures and analysis must be written independently on your own.

The Department uses code profiling tools such as Turnitin to check for plagiarism of any sort. **Plagiarism is a major form of academic misconduct.**

Marks

This coursework is worth **40% of your total mark for the course.**

Mastery component: The mastery component amounts to 20% of this CW: two tasks (**1.4 and 3.2**) have one version for 3rd year BSc students only and one version for MSc and 4th year MSci students only (Mastery component) - pay attention to this and choose the right option in each of those tasks! You should only answer the version that applies to your case.

For general guidance about writing your solutions and marking scheme, please read the document “Guidelines about writing your solutions” in the folder “Examples of Previous Coursework & Guidelines”.

Submission

Files: For the submission of your coursework, you need to save **two documents**:

- a **Jupyter notebook (file format: `ipynb`)** with all your tasks clearly labelled. You should use the template notebook called `CID_Coursework1.ipynb` provided on Blackboard (folder ‘Coursework/Coursework 1’). The notebook should have clear headings to indicate the answers to each question, e.g. ‘Task 1.1’.

The notebook should contain the cells with your code and their output, plus some brief text explaining your calculations, choices, mathematical reasoning, and discussion of results.

Important: Before submitting you must run the notebook, with all cells executed sequentially and the outputs of the cells must be printed. You can use Google Colab or develop your Jupyter notebook in the local Python environment installed on your computer.

- Once you have executed all cells in your notebook and their outputs are printed, save the notebook as an **html file** (with name *CID_Coursework1.html*). Your ipynb file must produce all the output that appears in your html file, *i.e.*, make sure you have run all cells in the notebook before exporting the html.

Trouble-shooting for conversion to HTML if you use Google Colab: as a default, Colab does not let you download your notebook as html. Please try one of the following: 1) press ctrl+S; 2) follow the easy instructions [here](#); 3) simply download your notebook through File > Download .ipynb and use the standard tool for Jupyter Notebook conversion nbconvert.

The submission of your coursework must consist of **two items**, uploaded **separately**:

- 1) A **single zip folder** containing your Jupyter notebook as an **ipynb file** and your notebook exported as an **html file**. Name your zip folder 'CID_Coursework1.zip', where CID is your student CID, e.g. 123456_Coursework1.zip.
- 2) The html file, named *CID_Coursework1.html*, which will be used for the plagiarism check.

The submission should consist only of these 2 items - **Do not submit multiple files.**

Do not put your name on the files you submit (only the CID), because the marking must be carried out preserving your anonymity.

Drop Boxes: The submission is done **online via Blackboard**, using the drop boxes inside the folder 'Coursework' on Blackboard.

- If you are a 3rd year BSc student: use the 'Coursework Drop Boxes' inside the folder 'Coursework'. The html file must be uploaded to 'Coursework 1/Coursework 1 Spring 26 - HTML', the zip folder must be uploaded to 'Coursework 1/Coursework 1 Spring 26 - ZIP File'.
- If you are a 4th year MSci or MSc student: use the 'Mastery Coursework Drop Boxes' inside the folder 'Coursework'. The html file must be uploaded to 'Mastery Coursework 1/Coursework 1 Mastery Spring 26 - HTML', the zip file to 'Mastery Coursework 1/Coursework 1 Mastery Spring 26 - ZIP File'.

Make sure you submit to the right drop box on Blackboard - any mistake in the submission folder will cause a delay in the release of your mark.

Recommendations about online submissions:

- There are known issues with particular browsers (or settings with cookies or popup blockers) when submitting to Turnitin. If the submission 'hangs', please try another browser.
- You should also **check that your files are not empty or corrupted after submission**.
- To avoid last minute problems with your online submission, we recommend that you **upload versions of your coursework early on**, before the deadline. You will be able to update your coursework until the deadline. Keep **backups of your work**, e.g., save regularly your notebook with its outputs as an .html file, which can be useful if something unpredicted happens just before the deadline.
- If you have any issue with the submission, or you realise you have submitted your work to the wrong drop box, please contact directly the UGMathsOffice at maths-student-office@imperial.ac.uk or your MSc programme administrator, in such a way that they can help you solve the issue.
- If you need an extension, or happen for any reason to submit your work late, please make a request for mitigating circumstances directly on ZINC.

For these last two points, do not contact us - we, as lecturers, are not able to grant extensions nor to make changes in the submission folders! We only get to see anonymised submissions.

Coursework 1

In this coursework, you will work with a dataset on **astronomical observations of asteroids**, which is made available inside the folder 'Coursework/Coursework 1/Data' on Blackboard. With this dataset you will perform supervised learning tasks of **regression** (Task 1 and 2) as well as **classification** (Task 3).

Dataset: The asteroid dataset contains information on asteroids' orbits, such as eccentricity and inclination, and other observational properties, such as asteroids' intrinsic brightness (absolute magnitude) and physical reflectivity (albedo). Specifically, it contains **7 numerical variables (or “data features”) to use as predictors in the tasks:** Absolute magnitude, Albedo, Number of observations, Observation arc length, Orbital eccentricity, Orbital inclination, Orbital period.

These data features can be leveraged to:

- Estimate additional, not directly observed properties of the asteroids' structure, such as their diameter. This dataset provides available values of the diameter (column **Asteroid diameter**) that you will use as the known outputs for regression tasks.
- Classify asteroids into astronomical classes, specifically: MBA (Main-Belt Asteroid), OMB (Outer Main-Belt), and TJN (Jupiter Trojan). The column **Asteroid class** provides known class labels of this type that you will use for the classification tasks.

The dataset to use as **the training set** consists of $N^{\text{train}} = 638$ samples and is available on Blackboard as: `asteroid_observations_train.csv`.

The dataset to use as **the test set** consists of $N^{\text{test}} = 160$ samples and is available on Blackboard as: `asteroid_observations_test.csv`.

Important:

1. The test set should **not** be used in any learning, specifically it should not be involved in any parameter training or cross-validation-based hyperparameter tuning. The test set should only be used a posteriori to evaluate the out-of-sample performance of your methods.
2. For reproducibility, create a random generator with a **fixed seed** at the beginning of your notebook with the command: `rng = np.random.default_rng(0)`.
3. Use standardised data for all the tasks, using mean and standard deviation of the training set to standardise both the training and validation/test sets, as discussed in Week 4 tutorial.

Task 1: Linear regression and feature selection

In this task, you will perform linear regression to predict the asteroid's diameter value (column **Asteroid diameter**) from the 7 data features and, based on this, you will implement strategies of feature selection to identify the most predictive data features.

1.1 (10 marks) – Garrote feature selection. Garrote is a linear-regression-based method for feature selection that was proposed before LASSO. Garrote aims at learning a linear regression model formulated as follows:

$$y = f_{\beta^G, \hat{\beta}_0^{LS}}(\mathbf{x}) + \epsilon \quad \text{with: } f_{\beta^G, \hat{\beta}_0^{LS}}(\mathbf{x}) = \mathbf{x}^T \beta^G + \hat{\beta}_0^{LS} \quad \text{and} \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

where $\mathbf{x}^T = (x_1, \dots, x_p)$ is the vector of $p = 7$ data features and y is the regression target (the asteroid's diameter). The Garrote coefficients $(\beta^G)^T = (\beta_1^G, \dots, \beta_p^G)$ are given by:

$$\beta_j^G = g_j \hat{\beta}_j^{LS} \quad j = 1, \dots, p \quad g_j \in \mathbb{R}$$

where $(\hat{\beta}^{LS})^T = (\hat{\beta}_0^{LS}, \hat{\beta}_1^{LS}, \dots, \hat{\beta}_p^{LS})$ are the ordinary least-squares parameter estimates for the linear regression function modelling the relationship between $\mathbf{x}$ and y . The Garrote parameters $\mathbf{g} = (g_1, \dots, g_p)$ are learnt by minimising a loss given by the mean squared error (MSE) over the training set, i.e., by taking:

$$\min_{\mathbf{g}} \left[\frac{1}{N^{\text{train}}} \sum_{i=1}^{N^{\text{train}}} \left(y^{(i)} - f_{\beta^G, \hat{\beta}_0^{LS}}(\mathbf{x}^{(i)}) \right)^2 \right]$$

under the constraint:

$$\|g\|_1 = \sum_{j=1}^p |g_j| < s \quad \text{for some } s \in \mathbb{R}_{\geq 0}$$

To perform this constrained minimisation: introduce the appropriate penalty term in the loss with hyperparameter λ ; use the Huber function-based approximation of the loss to ensure smoothness of its gradient (setting $c_{\text{huber}} = 1e - 4$); implement gradient descent with $n_{\text{iters}} = 10000$ and $\text{step_size} = 1e - 7$.

Train the Garrote method on the training set for all the values of λ in the set S_λ generated by the command `np.logspace(-1, 7.6, 15)`. Visualize the Garrote coefficient trajectories, i.e., plot the learnt coefficients β^G as a function of $\log_{10}(\lambda)$. To aid visualisation set the scale of the y -axis to `plt.yscale("symlog", llinthresh=0.1)`. Discuss your results in terms of Garrote's ability to perform feature selection.

1.2 (8 marks) – Assessing Garrote's stability via T-fold cross-validation. A predictor is said to be unstable if a small change in the training data can make large changes in its out-of-sample performance. To assess the effect on Garrote's performance of changes in the training data you will perform T-fold cross-validation. Split your training data into T folds, for $T = 2, 20$ (do not reshuffle the data before dividing them into folds). For all values of T and for all values of λ in the set S_λ from Task 1.1:

- Train Garrote, using the same c_{huber} , n_{iters} and step_size as in Task 1.1;
- Compute the average validation MSE, MSE_t , for all folds $t = 1, \dots, T$ (see Chapter 2 of the Lecture Notes);
- Compute the average absolute difference in the validation MSE between successive folds D :

$$D = \frac{1}{T-1} \sum_{t=2}^T |MSE_t - MSE_{t-1}|$$

Plot D as a function of $\log_{10}(\lambda)$ for each T (i.e., you should have a plot with 2 curves, one for each T). Use this plot to discuss the stability of Garrote as a function of the feature selection-enforcing hyperparameter λ , explaining also the reasons for any difference you might observe between different values of T .

1.3 (14 marks) – Assessing LASSO feature selection stability via bootstrap. A feature selection procedure is said to be unstable if a small change in the training data can make large changes in its set of selected features. In this subtask, you will train LASSO regression for feature selection and you will apply bootstrap to assess the stability of its selected features to changes in the training data, by computing the so-called "selection probability".

Specifically, in this subtask, solve the LASSO regression problem specified by the loss function:

$$L(\beta, \beta_0) = \frac{1}{N^{\text{train}}} \sum_{i=1}^{N^{\text{train}}} \left(y^{(i)} - f_{\beta, \beta_0}(\mathbf{x}^{(i)}) \right)^2 + \lambda \|\beta\|_1 \quad \|\beta\|_1 = \sum_{j=1}^p |\beta_j|$$

where:

$$f_{\beta, \beta_0}(\mathbf{x}) = \mathbf{x}^T \beta + \beta_0 \quad \text{with: } \beta^T = (\beta_1, \dots, \beta_p), \mathbf{x}^T = (x_1, \dots, x_p)$$

Implement LASSO with the Huber function-based smoothing using everywhere: $n_{\text{iters}} = 10000$; $\text{step_size} = 1e - 7$; $c_{\text{huber}} = 1e - 4$.

First, train LASSO on the training dataset for all values of λ in the set S_λ generated by the command `np.logspace(-1, 7.6, 15)`, and plot the LASSO coefficient trajectories as a function of $\log_{10}(\lambda)$. Compare your results to the ones from Garrote (Task 1.1).

Next, use the following bootstrap-based procedure to compute the selection probability of the different data features. For a given λ :

- Resample without replacement $B = 100$ bootstrap samples of size N' from the training dataset.
- For each bootstrap sample, train a LASSO regressor as described above and decide which features are selected by it (consider a feature selected if it has an associated coefficient > 0.001 in absolute value).
- For each feature, compute its selection probability as the fraction of bootstrap samples where that feature was selected through LASSO.

Implement this procedure for two choices of N' : $N' = N^{\text{train}}/2$ and $N' = 50$. For each choice of N' , compute the features' selection probabilities for all values of λ in S_λ and produce a plot of the selection probability

trajectories, i.e., the selection probabilities for the different features as a function of $\log_{10}(\lambda)$. For each N' , produce also a bar plot of the selection probabilities for the λ value with Python index 8 in the set S_λ .

Provide an interpretation of the selection probability in terms of feature selection stability. Use the plots you have produced to discuss the stability of the different data features selected through LASSO.

1.4 (7 marks, BSc students only) – Feature importance in linear regression. Train an ordinary linear regressor on the training data and estimate the relative importance of each feature through the method of variable importance explained in Chapter 5 of the Lecture Notes. Use the test set in place of the out-of-bag samples and the R^2 score as metric of performance. Produce a bar plot of variable importance for all features, expressing it as a percentage of the importance of the most important feature. Use this plot to discuss the relative importance of features, and connect these results to the above results of feature selection obtained through Garrote (Task 1.1) and LASSO (Task 1.3).

1.4 (9 marks, MSc/4th-year students only) – Dominance analysis in linear regression. In this subtask, you will estimate the relative importance of each feature in ordinary linear regression through the method known as “dominance analysis”. Dominance analysis is an approach to estimating the relative importance of predictors in regression problems that is presented in [this article](#).

In dominance analysis, the relative importance of a predictor is measured by the additional contribution to the in-sample R^2 score that results from adding that predictor to all subset models. Specifically, for a given feature x_j , the steps of dominance analysis you are asked to implement here are as follows:

- Compute all the possible non-empty feature subsets that do not contain x_j , let us call the collection of these subsets $S_{\setminus x_j}$ (e.g., $S_{\setminus x_1}$ contains: all possible subsets with one feature: $\{x_2\}, \{x_3\}, \dots, \{x_p\}$; all possible subsets with two features $\{x_2, x_3\}, \dots, \{x_{p-1}, x_p\}$, all possible subsets with three features, etc.).
- Add the feature x_j to each subset s in $S_{\setminus x_j}$ – let us denote these x_j -containing subsets $\{s, x_j\}$, with $s \in S_{\setminus x_j}$.
- For each feature subset $s \in S_{\setminus x_j}$, train two ordinary linear regressors on the training dataset: the first including the features in s , the second including the features in $\{s, x_j\}$. For both models compute the in-sample R^2 score, respectively denoted as $R^2_{\{s\}}$ and $R^2_{\{s, x_j\}}$.
- Compute the additional contribution of feature x_j , AC_{x_j} , by summing across feature subsets the x_j -induced gain in the in-sample R^2 score, i.e.:

$$AC_{x_j} = \sum_{s \in S_{\setminus x_j}} R^2_{\{s, x_j\}} - R^2_{\{s\}}$$

Report the number of linear models fitted for each feature x_j to compute AC_{x_j} , justifying your answer. Produce a bar plot of AC_{x_j} for all features x_j and use this plot to discuss the relative importance of features, and connect these results to the above results of feature selection obtained through Garrote (Task 1.1) and LASSO (Task 1.3).

Task 2: Non-linear regression with Kernel Ridge Regression

In this task you will train a non-linear regressor that predicts the outcome variable y (i.e., the asteroid diameter provided in the column **Asteroid diameter**) by implementing a kernelised version of Ridge regression.

2.1 (18 marks) – Kernel Ridge Regression. First, use the training data to fit the Ridge regressor with loss:

$$L(\beta, \beta_0) = \frac{1}{N^{\text{train}}} \sum_{i=1}^{N^{\text{train}}} \left(y^{(i)} - f_{\beta, \beta_0}(\mathbf{x}^{(i)}) \right)^2 + \lambda \|\beta\|^2 \quad \|\beta\|^2 = \sum_{j=1}^p |\beta_j|^2 \quad (1)$$

where:

$$f_{\beta, \beta_0}(\mathbf{x}) = \mathbf{x}^T \beta + \beta_0 \quad \text{with: } \beta^T = (\beta_1, \dots, \beta_p), \mathbf{x}^T = (x_1, \dots, x_p)$$

Set $\lambda = 0.1$. Evaluate the Ridge regressor's performance on the training and test set using the R^2 score as your metric of performance.

Next, formulate the kernelised version of the Ridge regression problem specified by the loss function (1) with $\beta_0 = 0$ (i.e., without the intercept term). Derive and write down explicitly the loss function of this kernelised version of Ridge regression, which we call Kernel Ridge Regression (KRR).

Train KRR on the training dataset by minimising analytically its loss, setting again $\lambda = 0.1$. Use as kernel the polynomial kernel k^P of degree up to n , which, for two generic vectors v and z , reads:

$$k^P(v, z) = (v \cdot z + c)^n \quad (2)$$

Train the KRR for 3 different combinations of the polynomial kernel's parameters: $n = 2, c = 0$; $n = 2, c = 1$; $n = 3, c = 1$. Report the training and test set R^2 score for each of these parameter combinations. Discuss your results making a direct comparison with ordinary Ridge regression (Equation 1) and explaining any difference in performance you might observe.

2.2 (12 marks) – Fitting power of different kernels. In this subtask, you will attempt to gain a visual intuition of the fitting power of different kernels and of the effect of the hyperparameter λ in KRR. To this end, restrict the data features to the first two (**Absolute magnitude** and **Albedo**), i.e., consider as inputs to the KRR model the reduced data vector:

$$\bar{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

. You will compare 3 different kernel choices:

- The polynomial kernel k^P given by equation (2) with $n = 2$ and $c = 1$;
- The radial basis function kernel k^{RBF} , which, for two generic vectors v and z , is defined as:

$$k^{RBF}(v, z) = e^{-\frac{\|v-z\|^2}{\sigma}}$$

- A new kernel k^{new} that you will construct starting from k^P and k^{RBF} setting:

$$k^{new}(v, z) = k^{RBF}(\phi^P(v), \phi^P(z)) \quad (3)$$

where $\phi^P(v)$ is the transformation associated with k^P defined in (2) with $n = 2$ and $c = 1$, i.e.:

$$\phi^P(v) \cdot \phi^P(z) = (v \cdot z + 1)^2$$

Set $\sigma = 20$ everywhere in the k^{RBF} kernel. For all 3 kernel choices: train KRR in the same way as you did in Task 2.1, setting $\lambda = 0.1$, and report the R^2 score on the training set; produce a 3D plot with the data features x_1 and x_2 along the x and y axes and the predicted outcome y along the z -axis to visualize the data fit of the KRR you have trained. Specifically, in this plot, visualize the training data points and the KRR regression function evaluated on the training data points. [Hint: for the latter you can use the method `plot_surface`.]

Repeat the KRR training, the evaluation of the training-set R^2 score and the 3D plots for all 3 kernel choices for $\lambda = 300$. Based on the R^2 scores and the 6 plots you have produced, draw conclusions on the fitting power of different kernels and the effect of λ in KRR.

Task 3: Classification with the Multi-Layer Perceptron

In this task, you will build a soft multi-class classifier that, based on the 7 data features, predicts the asteroid's probability of belonging to each of the classes stored in the column **Asteroid class**. To this end, consider class MBA as class label 0; class OMB as class label 1; and class TJN as class label 2.

You will build the classifier by implementing the Multi-Layer Perceptron (MLP). Use NumPy alone like in your Week 5 notebook, do not use TensorFlow/Keras, PyTorch or equivalent libraries. Create MLP objects from the MLP class provided by the Week 5 notebook, initialising them with `seed= 2` everywhere in this task. Initialise all the `sgd` routines with `seed= 42`.

3.1 (18 marks) – MLP for classification. Set up the MLP architecture for soft multi-class classification as follows. Your network should have an input layer, 3 hidden layers (with, respectively, 64, 64 and 32 neurons), followed by the output layer with 3 neurons (the $Q = 3$ class probabilities). For all the hidden layers, apply the ReLU activation function. Train the network on the training set minimising the cross-entropy (CE) function:

$$CE = - \sum_{i=1}^{N^{\text{train}}} \sum_{q=0}^{Q-1} y_q^{(i)} \log p_q^{(i)} \quad (4)$$

where $y_q^{(i)}$ is q^{th} element of the class label $y^{(i)}$ of sample i in one-hot encoding¹, while $p_q^{(i)}$ are the MLP's

¹In one-hot encoding, the class label 0 is expressed by the vector $y = (1, 0, 0)$; the class label 1 is expressed by the vector $y = (0, 1, 0)$,

predicted probabilities for sample i . They are given by the softmax function applied to the pre-activations $\mathbf{a}^{(N_L+1)}$ of the output units, i.e.:

$$p_q = \text{softmax}(\mathbf{a}^{(N_L+1)})_q = \frac{\exp(a_q^{(N_L+1)})}{\sum_{q'=0}^{Q-1} \exp(a_{q'}^{(N_L+1)})} \quad \text{for } Q = 3 \text{ and with } N_L : \text{Number of hidden layers} \quad (5)$$

Use mini-batch stochastic gradient descent (SGD) as your optimisation method, setting the mini-batch size = 64. Train the network for 500 epochs, with a constant learning rate equal to 0.04. Track the CE loss function (4) on the training and test set during training and plot it as a function of the number of epochs. Use this plot to comment on the training convergence and the model's performance on the training and test set.

Evaluate the performance of the trained MLP on the test set using accuracy and per-class accuracy as your metrics of performance. The per-class accuracy for a class q , ACC_q , is defined as:

$$ACC_q = \frac{\text{Number of correct predictions for data points of class } q}{\text{Number of data points of class } q}$$

For each class q , produce two count histograms: a count histogram of the MLP's predicted probabilities $p_q^{(i)}$ evaluated on all the test-set samples i of class q ; a count histogram of the MLP's predicted probabilities $p_q^{(i)}$ evaluated on all the test-set samples i of class $q' \neq q$. [Hint: you should obtain 6 plots]. Use the accuracy metrics and these plots to discuss the MLP out-of-sample performance across classes.

3.2 (13 marks, BSc students only) – Gradient Boosting MLPs. To attempt to maximise the MLP performance at discriminating asteroids' classes, you will implement gradient boosting with MLPs (GBMLP).

The strong model of the GBMLP is given by a soft multi-class MLP classifier which is trained to minimise the CE loss (Equation 4) where the predicted class probabilities are given by:

$$p_q = \text{softmax}(\mathbf{a})_q = \frac{\exp(a_q)}{\sum_{q'=0}^{Q-1} \exp(a_{q'})} \quad \text{for } Q = 3$$

Take $\mathbf{a} = (a_0, a_1, a_2)$ as the strong model's predictions to update iteratively through the weak models. Provide the iterative procedure with a "warm start" by initialising the strong model's predictions $\mathbf{a}$ to the values of $\mathbf{a}^{(N_L+1)}$ outputted by the MLP trained in Task 3.1.

Each weak model is an MLP with the following network architecture: an input layer, 3 hidden layers (with, respectively, 32, 32 and 16 hidden neurons), followed by the output layer with 3 neurons. The first two hidden layers apply the ReLU activation function, the last one applies a simple identity function. Each weak model is trained via minimisation of an MSE loss.

Train GBMLP with 4 weak models and the shrinkage hyperparameter set to $\nu = 2$, using mini-batch SGD with mini-batch size 32, 1000 training epochs, and a constant learning rate equal to 0.05. Evaluate GBMLP on the test set and report the class-specific accuracy after each iteration. Discuss your findings.

3.2 (11 marks, MSc/4th year students only) – Focal loss for multi-class classification. In this subtask, you will implement the focal loss, a special type of loss function to improve classification performance for hard-to-classify examples that was introduced in this article: [Focal Loss for Dense Object Detection](#).

Build an MLP for soft classification into the classes $q = 0, 1, 2$ that has the same network structure (in terms of number and width of layers, activation functions) as the MLP in Task 3.1. Here the MLP is trained by minimising the focal loss:

$$FL = \sum_{i=1}^{N^{\text{train}}} FL^{(i)}$$

where $FL^{(i)}$ is the focal loss for sample i and is given by a multi-class generalisation of Equation (4) in the focal loss [article](#) (attention: do not consider the class-specific weighting factor α introduced further on in the article). Provide an explicit expression of the multi-class focal loss FL for this subtask.

Set the exponent γ in FL to the value $\gamma = 2$. Train the MLP on the training set using the same mini-batch size, number of epochs and learning rate as in Task 3.1. Report the accuracy and per-class accuracy on the test set and produce the 6 count histograms of the MLP's predicted class probabilities like in Task 3.1.

Discuss your results, comparing them to the ones from Task 3.1 and explaining the reasons for any difference in performance you might observe. Comment explicitly on and explain the effects of the focal loss on the predicted class probabilities.

and the class label 2 is expressed by the vector $\mathbf{y} = (0, 0, 1)$. [Hint: to obtain a one-hot encoding you can use the `np.eye` method.]